

Chapter 8. Exceptions

Some operations can fail. If your program is reading data from a file stored on an external drive, someone might disconnect the drive. Your application might try to construct an array only to discover that the system does not have enough free memory. Intermittent wireless network connectivity can cause network requests to fail. One widely used way for a program to discover these sorts of failures is for each API to return a value indicating whether the operation succeeded. This requires developers to be vigilant if all errors are to be detected, because programs must check the return value of every operation. This is certainly a viable strategy, but it can obscure the code; the logical sequence of work to be performed when nothing goes wrong can get buried by all of the error checking, making the code harder to maintain. C# supports another popular error-handling mechanism that can mitigate this problem: *exceptions*.

When an API reports failure with an exception, this disrupts the normal flow of execution, leaping straight to the nearest suitable error-handling code. This enables a degree of separation between error-handling logic and the code that tries to perform the task at hand. This can make code easier to read and maintain, although it does have the downside of making it harder to see all the possible ways in which the code may execute.

Exceptions can also report problems with operations where a return code might not be practical. For example, the runtime can detect and report problems for basic operations, even something as simple as using a reference. Reference type variables can contain `null`, and if you try to invoke a method on a null reference, it will fail. The runtime reports this with an exception.

Most errors in .NET are represented as exceptions. However, some APIs offer you a choice between return codes and exceptions. For example, the `int` type has a `Parse` method that takes a string and attempts to inter-

pret its contents as a number, and if you pass it some nonnumeric text (e.g., "Hello"), it will indicate failure by throwing a `FormatException` . If you don't like that, you can call `TryParse` instead, which does exactly the same job, but if the input is nonnumeric, it returns `false` instead of throwing an exception. (Since the method's return value has the job of reporting success or failure, the method provides the integer result via an `out` parameter.) Numeric parsing is not the only operation to use this pattern, in which a pair of methods (`Parse` and `TryParse` , in this case) provides a choice between exceptions and return values. As you saw in [Chapter 5](#), dictionaries offer a similar choice. The indexer throws an exception if you use a key that's not in the dictionary, but you can also look up values with `TryGetValue` , which returns `false` on failure, just like `TryParse` . Although this pattern crops up in a few places, for the majority of APIs, exceptions are the only choice.

If you are designing an API that could fail, how should it report failure? Should you use exceptions, a return value, or both? Microsoft's class library design guidelines contain instructions that seem unequivocal:

Do not return error codes. Exceptions are the primary means of reporting errors in frameworks.

—.NET Framework Design Guidelines

But how does that square with the existence of `int.TryParse` ? The guidelines have a section on performance considerations for exceptions that says this:

Consider the Try-Parse pattern for members that might throw exceptions in common scenarios to avoid performance problems related to exceptions.

—.NET Framework Design Guidelines

Failing to parse a number is not necessarily an error. For example, you might want your application to allow the month to be specified numerically or as text. So there are certainly common scenarios in which the operation might fail, but the guideline has another criterion: it suggests using it for "extremely performance-sensitive APIs," so you should offer the

`TryParse` approach only when the operation is fast compared to the time taken to throw and handle an exception.

Exceptions can typically be thrown and handled in a few microseconds, so they're not desperately slow—not nearly as slow as reading data over a network connection, for example—but they're not blindingly fast either. I find that on my computer, a single thread can parse five-digit numeric strings at a rate of roughly 100 million strings per second on .NET 8.0, and it's capable of rejecting nonnumeric strings at a similar speed if I use `TryParse`. The `Parse` method handles numeric strings just as fast, but it's roughly 500 times slower at rejecting nonnumeric strings than `TryParse`, thanks to the cost of exceptions. Of course, converting strings to integers is a pretty fast operation, so this makes exceptions look particularly bad, but that's why this pattern is most common on operations that are naturally fast.

Exceptions can be especially slow when debugging. This is partly because the debugger has to decide whether to break in, but it's particularly pronounced with the first unhandled exception your program hits. This can give the impression that exceptions are considerably more expensive than they really are. The numbers in the preceding paragraph are based on observed runtime behavior without debugging overheads. That said, those numbers slightly understate the costs, because handling an exception tends to cause the CLR to run bits of code and access data structures it would not otherwise need to use, which can have the effect of pushing useful data out of the CPU's cache. This can cause code to run slower for a short while after the exception has been handled, until the nonexceptional code and data can make their way back into the cache. The simplicity of my example reduces this effect.

Most APIs do not offer a `TryXxx` form, and will report all failures as exceptions, even in cases where failure might be common. For example, the file APIs do not provide a way to open an existing file for reading without throwing an exception if the file is missing. (You can use a different API to test whether the file is there first, but that's no guarantee of success. It's always possible for some other process to delete the file between your asking whether it's there and attempting to open it.) Since filesystem operations are inherently slow, the `TryXxx` pattern would not offer a

worthwhile performance boost here even though it might make logical sense.

WARNING

If you do use the `TryXxx` pattern, be aware that in cases where there are multiple reasons the operation could fail, the `false` return value typically indicates just one particular kind of failure. So a method of this kind might still throw an exception for some failure modes.

Exception Sources

Class library APIs are not the only source of exceptions. They can be thrown in any of the following scenarios:

- Your own code detects a problem.
- Your program uses a class library API, which detects a problem (e.g., you're using a database client library, and the database rejects an attempt to modify a row because it violates a data integrity constraint).
- The runtime detects the failure of an operation (e.g., arithmetic overflow in a checked context, or an attempt to use a null reference, or an attempt to allocate an object for which there is not enough memory).
- The runtime detects a situation outside of your control that affects your code (e.g., the runtime tries to allocate memory for some internal purpose and finds that there is not enough free memory).

Although these all use the same exception-handling mechanisms, the places in which the exceptions emerge are different. When your own code throws an exception (which I'll show you how to do later), you'll know what conditions cause it to happen, but when do these other scenarios produce exceptions? I'll describe where to expect each sort of exception in the following sections.

Exceptions from APIs

With an API call, there are several kinds of problems that could result in exceptions. You may have provided arguments that make no sense, such

as a `null` reference where a non-null one is required, or an empty string where the name of a file was expected. Or the arguments might look OK individually but not collectively. For example, you could call an API that copies data into an array, asking it to copy more data than will fit. You could describe these as “that will never work”-style errors, and they are usually the result of mistakes in the code. (One developer who used to work on the C# compiler team refers to these as *boneheaded* exceptions.)

A different class of problems arises when the arguments all look plausible but the operation turns out not to be possible given the current state of the world. For example, you might ask to open a particular file, but the file may not be present; or perhaps it exists, but some other program already has it open and has demanded exclusive access to the file. Yet another variation is that things may start well but conditions can change, so perhaps you opened a file successfully and have been reading data for a while, but then the file becomes inaccessible. As suggested earlier, someone may have unplugged a disk, or the drive could have failed due to overheating or age.

Software that communicates with external services over a network needs to take into account that an exception doesn’t necessarily indicate that anything is really wrong—sometimes requests fail due to some temporary condition, and you may just need to retry the operation. This is particularly common in cloud environments, where individual servers often come and go as part of the load balancing that cloud platforms typically offer—it is normal for a few operations to fail for no particular reason.

TIP

When using services via a library, you should find out whether it already handles this for you. For example, the Azure Storage libraries perform retries automatically by default and will only throw an exception if you disable this behavior or if problems persist after several attempts. You shouldn’t normally add your own exception handling and retry loops for this kind of error around libraries that do this for you.

Asynchronous programming adds yet another variation. In Chapters [16](#) and [17](#), I’ll show various asynchronous APIs—ones where work can

progress after the method that started it has returned. Work that runs asynchronously can also fail asynchronously, in which case the library might have to wait until your code next calls into it before it can report the error.

Despite the variations, in all these cases the exception will come from some API that your code calls. (Even when asynchronous operations fail, exceptions emerge either when you try to collect the result of an operation or when you explicitly ask whether an error has occurred.)

[Example 8-1](#) shows some code where exceptions of this kind could emerge.

Example 8-1. Getting an exception from a library call

```
static void Main(string[] args)
{
    using (var r = new StreamReader(@"C:\Temp\File.txt"))
    {
        while (!r.EndOfStream)
        {
            Console.WriteLine(r.ReadLine());
        }
    }
}
```

There's nothing categorically wrong with this code, so we won't get any exceptions complaining about arguments being self-evidently wrong. (In the unofficial terminology, it makes no boneheaded mistakes.) If your computer's *C:* drive has a *Temp* folder, and if that contains a *File.txt* file, and if the user running the program has permission to read that file, and if nothing else on the computer has already acquired exclusive access to the file, and if there are no problems—such as disk corruption—that could make any part of the file inaccessible, and if no new problems (such as the drive catching fire) develop while the program runs, this code will work just fine: it will show each line of text in the file. But that's a lot of *ifs*.

If there is no such file, the `StreamReader` constructor will not complete. Instead, it will throw an exception. This program makes no attempt to

handle that, so the application would terminate. If you ran the program outside of Visual Studio's debugger, you would see the following output:

```
Unhandled exception. System.IO.FileNotFoundException: Could not find file 'C:\Temp\File.txt'.
File name: 'C:\Temp\File.txt'
   at Microsoft.Win32.SafeHandles.SafeFileHandle.CreateFile(String fullPath, FileMode mode, FileAccess access, FileShare share, FileOptions options)
   at Microsoft.Win32.SafeHandles.SafeFileHandle.Open(String fullPath, FileMode mode, FileAccess access, FileShare share, FileOptions options, Int64 preallocationSize)
   at System.IO.Strategies.OSFileStreamStrategy..ctor(String path, FileMode mode, FileAccess access, FileShare share, FileOptions options, Int64 preallocationSize)
   at System.IO.Strategies.FileStreamHelpers.ChooseStrategyCore(String path, FileMode mode, FileAccess access, FileShare share, FileOptions options, Int64 preallocationSize)
   at System.IO.Strategies.FileStreamHelpers.ChooseStrategy(FileStream fileStream, String path, FileMode mode, FileAccess access, FileShare share, Int32 bufferSize, FileOptions options, Int64 preallocationSize)
   at System.IO.StreamReader.ValidateArgsAndOpenPath(String path, Encoding encoding, Int32 bufferSize)
   at System.IO.StreamReader..ctor(String path)
   at Exceptional.Program.Main(String[] args) in c:\Examples\Ch08\Example1\Program.cs:line 10
```

This tells us what error occurred, and shows the full call stack of the program at the point at which the problem happened. On Windows, the system-wide error handling will also step in, so depending on how your computer is configured, you might see its error reporting dialog, and it may even report the crash to Microsoft's error reporting service. If you run the same program in a debugger, it will tell you about the exception and highlight the line on which the error occurred, as [Figure 8-1](#) shows.

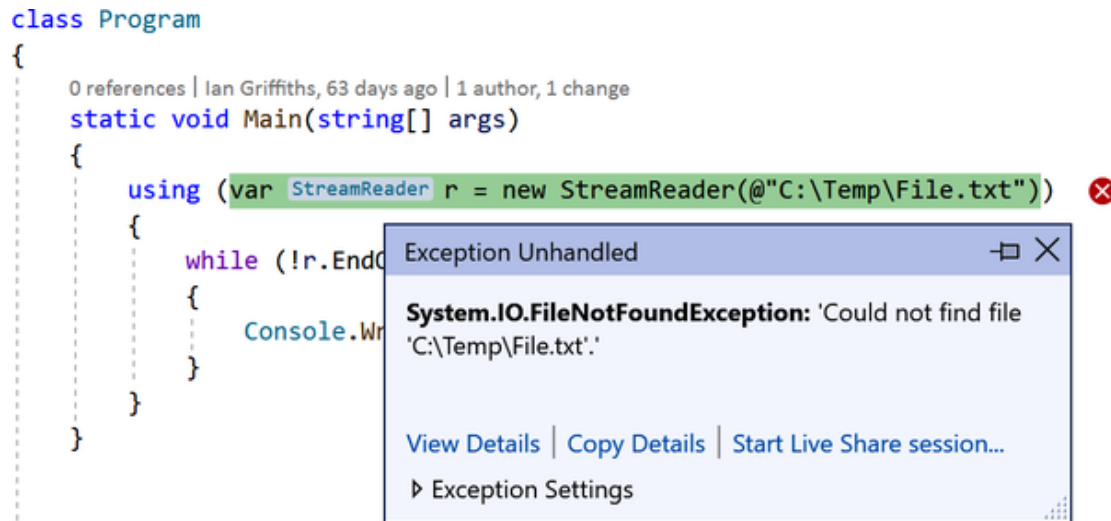


Figure 8-1. Visual Studio reporting an exception

What we’re seeing here is the default behavior that occurs when a program does nothing to handle exceptions: if a debugger is attached, it will step in, and if not, the program just crashes. I’ll show how to handle exceptions soon, but this illustrates that you cannot simply ignore them.

The call to the `StreamReader` constructor is not the only line that could throw an exception in [Example 8-1](#), by the way. The code calls `ReadLine` multiple times, and any of those calls could fail. In general, any member access could result in an exception, even just reading a property, although class library designers usually try to minimize the extent to which properties throw exceptions. If you make an error of the “that will never work” (boneheaded) kind, then a property might throw an exception but usually not for errors of the “this particular operation didn’t work” kind. For example, the documentation states that the `EndOfStream` property used in [Example 8-1](#) would throw an exception if you tried to read it after having called `Dispose` on the `StreamReader` object—an obvious coding error—but if there are problems reading the file, `StreamReader` will throw exceptions only from methods or the constructor.

Failures Detected by the Runtime

Another source of exceptions is when the CLR itself detects that some operation has failed. [Example 8-2](#) shows a method in which this could happen. As with [Example 8-1](#), there’s nothing innately wrong with this code (other than not being very useful). It is perfectly possible to use this with-

out causing problems. However, if someone passes in `0` as the second argument, the code will attempt an illegal operation.

Example 8-2. A potential runtime-detected failure

```
static int Divide(int x, int y)
{
    return x / y;
}
```

The CLR will detect when this division operation attempts to divide by zero and will throw a `DivideByZeroException`. This will have the same effect as an exception from an API call: if the program makes no attempt to handle the exception, it will crash, or the debugger will break in.

NOTE

Division by zero is not always illegal in C#. Floating-point types support special values representing positive and negative infinity, which is what you get when you divide a positive or negative value by zero; if you divide zero by itself, you get the special Not a Number value. None of the integer types support these special values, so integer division by zero is always an error.

The final source of exceptions I described earlier is also the detection of certain failures by the runtime, but they work a bit differently. They are not necessarily triggered directly by anything that your code did on the thread on which the exception occurred. These are sometimes referred to as *asynchronous exceptions*, and in theory they can be thrown at literally any point in your code, making it hard to ensure that you can deal with them correctly. However, these tend to be thrown only in fairly catastrophic circumstances, often when your program is about to be shut down, so you can't normally handle them in a useful way. For example, `StackOverflowException` and `OutOfMemoryException` can in theory be thrown at any point (because the CLR may need to allocate memory for its own purposes even if your code didn't do anything that explicitly attempts this).

I've described the usual situations in which exceptions are thrown, and you've seen the default behavior, but what if you want your program to do something other than crash?

Handling Exceptions

When an exception is thrown, the CLR looks for code to handle the exception. The default exception-handling behavior comes into play only if there are no suitable handlers anywhere on the entire call stack. To provide a handler, we use C#'s `try` and `catch` keywords, as [Example 8-3](#) shows.

Example 8-3. Handling an exception

```
try
{
    using (var r = new StreamReader(@"C:\Temp\File.txt"))
    {
        while (!r.EndOfStream)
        {
            Console.WriteLine(r.ReadLine());
        }
    }
}
catch (FileNotFoundException)
{
    Console.WriteLine("Couldn't find the file");
}
```

The block immediately following the `try` keyword is usually known as a *try block*, and if the program throws an exception while it's inside such a block, the CLR looks for matching *catch blocks*. [Example 8-3](#) has just a single `catch` block, and in the parentheses following the `catch` keyword, you can see that this particular block is intended to handle exceptions of type `FileNotFoundException`.

You saw earlier that if there is no `C:\Temp\File.txt` file, the `StreamReader` constructor throws a `FileNotFoundException`. In [Example 8-1](#), that caused our program to crash, but because [Example 8-3](#) has a `catch` block

for that exception, the CLR will run that `catch` block. At this point, it will consider the exception to have been handled, so the program does not crash. Our `catch` block is free to do whatever it wants, and in this case, my code just displays a message indicating that it couldn't find the file.

Exception handlers do not need to be in the method in which the exception originated. The CLR walks up the stack until it finds a suitable handler. If the failing `StreamReader` constructor call were in some other method that was called from inside the `try` block in [Example 8-3](#), our `catch` block would still run (unless that method provided its own handler for the same exception).

Exception Objects

Exceptions are objects, and their type derives from the `Exception` base class.¹ This defines properties providing information about the exception, and some derived types add properties specific to the problem they represent. Your `catch` block can get a reference to the exception if it needs information about what went wrong. [Example 8-4](#) shows a modification to the `catch` block from [Example 8-3](#). In the parentheses after the `catch` keyword, as well as specifying the exception type, we also provide an identifier (`x`) with which code in the `catch` block can refer to the exception object. This enables the code to read a property specific to the `FileNotFoundException` class: `FileName`.

Example 8-4. Using the exception in a `catch` block

```
try
{
    // ...same code as Example 8-3...
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"File '{ex.FileName}' is missing");
}
```

This will display the name of the file that couldn't be found. With this simple program, we already knew which file we were trying to open, but

you could imagine this property being helpful in a more complex program that deals with multiple files.

The general-purpose members defined by the base `Exception` class include the `Message` property, which returns a string containing a textual description of the problem. The default error handling for console applications displays this. The text `Could not find file 'C:\Temp\File.txt'` that we saw when first running [Example 8-1](#) came from the `Message` property. So I could have just displayed that instead of building my own message, although there will often be cases where your knowledge about what the code was trying to do when the problem occurs will enable you to produce a more informative message than the default. But the `Message` property is important when you're diagnosing unexpected exceptions.

WARNING

The `Message` property is intended for human consumption. It is therefore a bad idea to write code that attempts to interpret an exception by inspecting the `Message` property.

One reason you shouldn't rely on exact text appearing in `Message` is that some APIs localize these messages, so string comparisons could fail when your code runs on a computer configured to run in a region where the main spoken language is different than yours. Moreover, Microsoft doesn't treat exception message changes as breaking changes, so the text might change even within the same locale. It is best to rely on the actual exception type, although some exceptions such as `IOException` get used in ambiguous ways. So you sometimes need to inspect the `HResult` property, which will be set to an error code from the OS in such cases.

`Exception` also defines an `InnerException` property. This is often `null`, but it comes into play when one operation fails as a result of some other failure. Sometimes, exceptions that occur deep inside a library would make little sense if they were allowed to propagate all the way up to the caller. For example, .NET provides a library for parsing XAML files. (XAML—Extensible Application Markup Language—is used by various

.NET UI frameworks, including WPF.) XAML is extensible, so it's possible that your code (or perhaps some third-party code) will run as part of the process of loading a XAML file, and this extension code could fail—suppose a bug in your code causes an `IndexOutOfRangeException` to be thrown while trying to access an array element. It would be somewhat mystifying for that exception to emerge from a XAML API, so regardless of the underlying cause of the failure, the library throws a `XamlParseException`. This means that if you want to handle the failure to load a XAML file, you know exactly which exception to handle, but the underlying cause of the failure is not lost: when some other exception caused the failure, it will be in the `InnerException`.

All exceptions contain information about where the exception was thrown. The `StackTrace` property provides the call stack as a string. As you've already seen, the default exception handler for console applications displays that. There's also a `TargetSite` property, which tells you which method was executing. It returns an instance of the reflection API's `MethodBase` class. See [Chapter 13](#) for details on reflection.

Multiple catch Blocks

A `try` block can be followed by multiple `catch` blocks. If the first `catch` does not match the exception being thrown, the CLR will then look at the next one, then the next, and so on. [Example 8-5](#) supplies handlers for `FileNotFoundException`, `DirectoryNotFoundException`, and `IOException`.

Example 8-5. Handling multiple exception types

```
try
{
    using (var r = new StreamReader(@"C:\Temp\File.txt"))
    {
        while (!r.EndOfStream)
        {
            Console.WriteLine(r.ReadLine());
        }
    }
}
```

```

catch (FileNotFoundException ex)
{
    Console.WriteLine($"File '{ex.FileName}' is missing");
}
catch (DirectoryNotFoundException)
{
    Console.WriteLine($"The containing directory does not exist.");
}
catch (IOException ex)
{
    Console.WriteLine($"IO error: '{ex.Message}'");
}

```

An interesting feature of this example is that both `FileNotFoundException` and `DirectoryNotFoundException` derive from `IOException`. I could remove the first two `catch` blocks, and this would still handle these exceptions correctly (just with less-specific messages), because the CLR considers a `catch` block to be a match if it handles the base type of the exception. So [Example 8-5](#) has two viable handlers for a `FileNotFoundException` and also two viable handlers for `DirectoryNotFoundException`. (The third handler is still useful because the documentation tells us that for certain kinds of failure, `StreamReader` will throw an `IOException`, and not either of the more specific types.) In these cases, C# requires more specific handlers to come first. If I were to move the `IOException` handler above the other handlers, I'd get this compiler error for each of the more specific handlers:

```

error CS0160: A previous catch clause already catches all exceptions of this or
of a super type ('IOException')

```

If you write a `catch` block for the `Exception` base type, it will catch all exceptions. In most cases, this is the wrong thing to do. While it's good to handle the exceptions you can anticipate, if you don't know what an exception represents, you should normally let it pass. Otherwise, you risk masking a problem. If you let the exception carry on, it's more likely to get to a place where it will be noticed, increasing the chances that you will fix the problem properly at some point. A catchall handler would be appropriate if you intend to wrap all exceptions in another exception and throw that, like the `XamlParseException` described earlier. A catchall ex-

ception handler might also make sense if it's at a point where the only place left for the exception to go is the default handling supplied by the system. (That might mean the `Main` method for a console application, but for multithreaded applications, it might mean the code at the top of a newly created thread's stack.) It might be appropriate in these locations to catch all exceptions and write the details to a logfile or some similar diagnostic mechanism. Even then, once you've logged it, you would probably want to rethrow the exception, as described later in this chapter, or even terminate the process with a nonzero exit code.

WARNING

For critically important services, you might be tempted to write code that swallows the exception so that your application can limp on. This is a bad idea. If an exception you did not anticipate occurs, your application's internal state may no longer be trustworthy, because your code might have been halfway through an operation when the failure occurred. If you cannot afford for the application to go offline, the best approach is to arrange for it to restart automatically after a failure. Most cloud providers' hosting services can be configured to do this automatically.

Exception Filters

You can make a `catch` block conditional: if you provide an *exception filter* for your `catch` block, it will only catch exceptions when the filter condition is true. [Example 8-6](#) shows how this can be useful. It uses the client API for Azure Table Storage, a NoSQL storage service offered as part of Microsoft's Azure cloud computing platform. This API's `TableClient` class has an `AddEntity` method that will throw a `RequestFailedException` if something goes wrong. The problem is that "something goes wrong" is very broad and covers more than connectivity and authentication failures. You will also see this exception for situations such as an attempt to insert a row when another row with the same keys already exists. That is not necessarily an error—it can occur as part of normal usage in some optimistic concurrency models.

Example 8-6. `catch` block with exception filter

```
public static bool InsertIfDoesNotExist(MyEntity item, TableClient table)
{
    try
    {
        table.AddEntity(item);
        return true;
    }
    catch (RequestFailedException ex)
    when (x.Status == 409)
    {
        return false;
    }
}
```

[Example 8-6](#) looks for that specific failure case and returns `false` instead of allowing the exception to continue propagating up the stack. It does this with a `when` clause containing a filter, which must be an expression of type `bool`. If the `Execute` method throws a `StorageException` that does not match the filter condition, the exception will propagate as usual—it will be as though the `catch` block were not there.

TIP

When using exception filters, a single `try` block can have multiple `catch` blocks for the same exception. Normally that would cause a compiler error, because only the first such `catch` would do anything, but with filters, that's not necessarily the case, so the compiler allows it. You can even have one unfiltered `catch` for a particular exception type when there are also filtered `catch` blocks for the same type, but the unfiltered one must appear last.

An exception filter must be an expression that produces a `bool`. It can invoke external methods if necessary. [Example 8-6](#) just fetches a property and performs a comparison, but you are free to invoke any method as part of the expression.² However, you should be careful to avoid doing anything in your filter that might cause another exception. If that happens, that second exception will be lost.

Nested try Blocks

If an exception occurs in a `try` block that does not provide a suitable handler, the CLR will keep looking. It will walk up the stack if necessary, but you can have multiple sets of handlers in a single method by nesting one `try/catch` inside another `try` block, as [Example 8-7](#) shows.

`ShowFirstLineLength` nests a `try/catch` pair inside the `try` block of another `try/catch` pair. Nesting can also be done across methods—the `Main` method will catch any `NullReferenceException` that emerges from the `ShowFirstLineLength` method (which will be thrown if the file is completely empty—the call to `ReadLine` will return `null` in that case).

Example 8-7. Nested exception handling

```
static void Main(string[] args)
{
    try
    {
        ShowFirstLineLength(@"C:\Temp\File.txt");
    }
    catch (NullReferenceException)
    {
        Console.WriteLine("NullReferenceException");
    }
}

static void ShowFirstLineLength(string fileName)
{
    try
    {
        using (var r = new StreamReader(fileName))
        {
            try
            {
                Console.WriteLine(r.ReadLine()!.Length);
            }
            catch (IOException ex)
            {
                Console.WriteLine("Error while reading file: {0}",
                                   x.Message);
            }
        }
    }
}
```

```
    }  
    catch (FileNotFoundException ex)  
    {  
        Console.WriteLine("Couldn't find the file '{0}'", x.FileName);  
    }  
}
```

I nested the `IOException` handler here to make it apply to one particular part of the work: it handles only errors that occur while reading the file after it has been opened successfully. It might sometimes be useful to respond to that scenario differently than for an error that prevented you from opening the file in the first place.

The cross-method handling here is somewhat contrived. The `NullReferenceException` could be avoided by testing the return value of `ReadLine` for `null`. (The compiler would have pointed that out to me if I hadn't deliberately silenced it with the `!` after `ReadLine()`.) However, the underlying CLR mechanism this illustrates is extremely important. A particular `try` block can define `catch` blocks just for those exceptions it knows how to handle, allowing others to escape up to higher levels.

Letting exceptions carry on up the stack is often the right thing to do. Unless there is something useful your method can do in response to discovering an error, it's going to need to let its caller know there's a problem, so unless you want to wrap the exception in a different kind of exception, you may as well let it through.

NOTE

If you're familiar with Java, you may be wondering if C# has anything equivalent to checked exceptions. It does not. Methods do not formally declare the exceptions they throw, so there's no way the compiler can tell you if you have failed either to handle them or declare that your method might, in turn, throw them.

You can also nest a `try` block inside a `catch` block. This is important if there are ways in which your error handler itself can fail. For example, if your exception handler logs information about a failure to disk, that could fail if there's a problem with the disk.

Some `try` blocks never catch anything. It's illegal to write a `try` block that isn't followed directly by something, but that something doesn't have to be a `catch` block: it can be a *finally block*.

finally Blocks

A `finally` block contains code that always runs once its associated `try` block has finished. It runs whether execution left the `try` block simply by reaching the end, returning from the middle, or throwing an exception. The `finally` block will run even if you use a `goto` statement to jump right out of the `try` block. [Example 8-8](#) shows a `finally` block in use.

Example 8-8. A `finally` block

```
using Microsoft.Office.Interop.PowerPoint;

...

[STAThread]
static void Main(string[] args)
{
    var pptApp = new Application();
    Presentation pres = pptApp.Presentations.Open(args[0]);
    try
    {
        ProcessSlides(pres);
    }
    finally
    {
        pres.Close();
    }
}
```

This is an excerpt from a utility I wrote to process the contents of a Microsoft Office PowerPoint file. This just shows the outermost code; I've omitted the actual detailed processing code, because it's not relevant here (although if you're curious, the full version in the downloadable examples for this book exports animated slides as video clips). I'm showing it

because it uses `finally`. This example uses COM interop to control the PowerPoint application. This example closes the document once it has finished, and the reason I put that code in a `finally` block is that I don't want the program to leave things open if something goes wrong partway through. This is important because of the way COM automation works. It's not like opening a file, where the OS automatically closes everything when the process terminates. If this program exits suddenly, PowerPoint will not close whatever had been opened—it just assumes that you meant to leave things open. (You might do this deliberately when creating a new document that the user will then edit.) I don't want that, and closing the file in a `finally` block is a reliable way to avoid it.

Normally, you'd write a `using` statement for this sort of thing, but PowerPoint's COM-based automation API doesn't support .NET's `IDisposable` interface. In fact, as we saw in the previous chapter, the `using` statement works in terms of `finally` blocks under the covers, as does `foreach`, so you're relying on the exception-handling system's `finally` mechanism even when you write `using` statements and `foreach` loops.

NOTE

`finally` blocks run correctly when your exception blocks are nested. If some method throws an exception that is handled by a method that's, say, five levels above it in the call stack, and if some of the methods in between were in the middle of `using` statements, `foreach` loops, or `try` blocks with associated `finally` blocks, all of these intermediate `finally` blocks (whether explicit or generated implicitly by the compiler) will execute before the handler runs.

Handling exceptions is only half of the story, of course. Your code may well detect problems, and exceptions may be an appropriate mechanism for reporting them.

Throwing Exceptions

Throwing an exception is very straightforward. You simply construct an exception object of the appropriate type, and then use the `throw` key-

word. [Example 8-9](#) does this when its `position` argument is outside the range that makes sense.

Example 8-9. Throwing an exception

```
public static string GetCommaSeparatedEntry(string text, int position)
{
    string[] parts = text.Split(',');
    if (position < 0 || position >= parts.Length)
    {
        throw new ArgumentOutOfRangeException(nameof(position));
    }
    return parts[position];
}
```

The CLR does all of the work for us. It captures the information required for the exception to be able to report its location through properties like `StackTrace` and `TargetSite`. (It doesn't calculate their final values, because these are relatively expensive to produce. It just makes sure that it has the information it needs to be able to produce them if asked.) It then hunts for a suitable `try / catch` block, and if any `finally` blocks need to be run, it'll execute those.

[Example 8-9](#) illustrates a common technique used when throwing exceptions that report a problem with a method argument. Exceptions such as `ArgumentNullException`, `ArgumentOutOfRangeException`, and their base class `ArgumentException` can all report the name of the offending argument. (This is optional because sometimes you need to report inconsistency across multiple arguments, in which case there isn't a single argument to be named.) It's a good idea to use C#'s `nameof` operator. You can use this with any expression that refers to a named item, such as an argument, a variable, a property, or a method. It compiles into a string containing the item's name.

I could have simply used the string literal `"position"` here instead, but the advantages of `nameof` are that it can avoid silly mistakes (if I type `positon` instead of `position`, the compiler will tell me that there's no such symbol), and it can help avoid problems caused when renaming a symbol. If I were to rename the `position` argument in [Example 8-9](#), I

could easily forget to change a string literal to match. But by using `nameof(position)`, I'll get an error if I change the name of the argument to, say, `pos`, without also changing `nameof(position)`—the compiler will report that there is no identifier called `position`. If I ask a C#-aware IDE (e.g., Visual Studio or JetBrains Rider) to rename the argument, it will automatically update all the places in the code that use the symbol, so it will replace the exception's constructor argument with `nameof(input)` for me.

We could use a similar technique with `ArgumentNullException`, but .NET offers a helper function that can simplify throwing this particular exception. As [Example 8-10](#) shows, instead of having to write an `if` statement that tests the input, with a body that throws an exception identifying the correct parameter name, we can just call `ArgumentNullException.ThrowIfNull`.

Example 8-10. Throwing an `ArgumentNullException`

```
public static int CountCommas(string input)
{
    ArgumentNullException.ThrowIfNull(input);
    return input.Count(ch => ch == ',');
}
```

This tests whatever argument you pass and throws an `ArgumentNullException` if it is null. But how can this set the parameter name correctly? This `ThrowIfNull` method is annotated with the `CallerArgumentExpression` attribute. As [Chapter 14](#) describes, this attribute enables the `ThrowIfNull` helper to discover the text of the expression that the caller used as the argument. Since we pass our `input` argument to this helper, it will be passed an additional hidden argument, the string `"input"`. So this has all the same benefits as using `nameof` with other argument exceptions, but it also performs the relevant test for us.

Many exception types provide a constructor overload that lets you set the `Message` text. A more specialized message may make problems easier to diagnose, but there's one thing to be careful of. Exception messages often

find their way into diagnostic logs and may also be sent automatically in emails by monitoring systems.

WARNING

You should be careful about what information you put in exception messages. This is particularly important if your software will be used in countries with data protection laws such as the General Data Protection Regulation (GDPR) that applies in EU nations. If an exception message contains information that refers in any way to a specific user, and this ends up in a log, this might contravene those laws.

Rethrowing Exceptions

Sometimes it is useful to write a `catch` block that performs some work in response to an error but allows the error to continue once that work is complete. There's an obvious but wrong way to do this, illustrated in [Example 8-11](#).

Example 8-11. How not to rethrow an exception

```
try
{
    DoSomething();
}
catch (IOException ex)
{
    LogIOError(ex);
    // This next line is BAD!
    throw x; // Do not do this
}
```

This will compile without errors, and it will even appear to work, but it has a serious problem: it loses the context in which the exception was originally thrown. The CLR treats this as a brand-new exception (even though you're reusing the exception object) and will reset the location information: the `StackTrace` and `TargetSite` will report that the error originated inside your `catch` block. This could make it hard to diagnose

the problem, because you won't be able to see where it was originally thrown. [Example 8-12](#) shows how you can avoid this problem.

Example 8-12. Rethrowing without loss of context

```
try
{
    DoSomething();
}
catch (IOException ex)
{
    LogIOError(ex);
    throw;
}
```

The only difference between this and [Example 8-11](#) (aside from removing the warning comments) is that I'm using the `throw` keyword without specifying which object to use as the exception. You're allowed to do this only inside a `catch` block, and it rethrows whichever exception the `catch` block was in the process of handling. This means that the `Exception` properties that report the location from which the exception was thrown will still refer to the original throw location, not the rethrow.

There is another context-related issue to be aware of when handling exceptions that you might need to rethrow that arises from how the CLR supplies information to Windows Error Reporting (WER), the component that leaps into action when an application crashes on Windows.

Depending on how your machine is configured, WER might show a crash dialog that can offer options including restarting the application, reporting the crash to Microsoft, debugging the application, or just terminating it. In addition to all that, when a Windows application crashes, WER captures several pieces of information to identify the crash location. For .NET applications, this includes the name, version, and timestamp of the component that failed, the exception type that was thrown, and information about the location from which the exception was thrown. These pieces of information are sometimes referred to as the *bucket* values. If the application crashes twice with the same values, those two crashes go into the same bucket, meaning that they are considered to be in some sense the same crash.

Retrieving this information from the Windows Event Log is all very well for code running on computers you control (or you might prefer to use more direct ways to monitor such applications, using systems such as Microsoft's Application Insights to collect telemetry, in which case WER is not very interesting). Where WER becomes more important is for applications that may run on other computers outside of your control, e.g., applications with a UI that run entirely locally or console applications.

Computers can be configured to upload crash reports to an error reporting service, and usually, just the bucket values get sent, although the services can request additional data if the end user consents. Bucket analysis can be useful when deciding how to prioritize bug fixes: it makes sense to start with the largest bucket, because that's the crash your users are seeing most often. (Or, at least, it's the one seen most often by users who have not disabled crash reporting. I always enable this on my computers, because I want the bugs I encounter in the programs I use to be fixed first.)

NOTE

The way to get access to accumulated crash bucket data depends on the kind of application you're writing. For a line-of-business application that runs only inside your enterprise, you will probably want to run an error reporting server of your own, but if the application runs outside of your administrative control, you can use Microsoft's own crash servers. There's a certificate-based process for verifying that you are entitled to the data, but once you've jumped through the relevant hoops, Microsoft will show you all reported crashes for your applications, sorted by bucket size.

Certain exception-handling tactics can defeat the crash bucket system. If you write common error-handling code that gets involved with all exceptions, there's a risk that WER will think that your application only ever crashes inside that common handler, which would mean that crashes of all kinds would go into the same bucket. This is not inevitable, but to avoid it, you need to understand how your exception-handling code affects WER crash bucket data.

If an exception rises to the top of the stack without being handled, WER will get an accurate picture of exactly where the crash happened, but

things may go wrong if you catch an exception before eventually allowing it (or some other exception) to continue up the stack. A bit surprisingly, .NET will successfully preserve the location for WER even if you use the bad approach shown in [Example 8-11](#). (It's only from .NET's perspective inside that application that this loses the exception context—`StackTrace` will show the rethrow location. So WER does not necessarily report the same crash location as .NET code will see in the exception object.) It's a similar story when you wrap an exception as the `InnerException` of a new one: .NET will use that inner exception's location for the crash bucket values.

This means that it's relatively easy to preserve the WER bucket. The only ways to lose the original context are either to handle the exception completely (i.e., not to crash) or to write a `catch` block that handles the exception and then throws a new one without passing the original one in as an `InnerException`.

Although [Example 8-12](#) preserves the original context, this approach has a limitation: you can rethrow the exception only from inside the block in which you caught it. But asynchronous programming is now very prevalent, so it is common for exceptions to occur on some random worker thread. We need a reliable way to capture the full context of an exception, and to be able to rethrow it with that full context some arbitrary amount of time later, possibly from a different thread.

The `ExceptionDispatchInfo` class solves these problems. If you call its static `Capture` method from a `catch` block, passing in the current exception, it captures the full context, including the information required by WER. The `Capture` method returns an instance of `ExceptionDispatchInfo`. When you're ready to rethrow the exception, you can call this object's `Throw` method, and the CLR will rethrow the exception with the original context fully intact. Unlike the mechanism shown in [Example 8-12](#), you don't need to be inside a `catch` block when you rethrow. You don't even need to be on the thread from which the exception was originally thrown.

NOTE

If you use the `async` and `await` keywords described in [Chapter 17](#), they use `ExceptionDispatchInfo` for you to ensure that exception context is preserved correctly.

Failing Fast

Some situations call for drastic action. If you detect that your application is in a hopelessly corrupt state, throwing an exception may not be sufficient, because there's always the chance that something may handle it and then attempt to continue. This risks corrupting persistent state—perhaps the invalid in-memory state could lead to your program writing bad data into a database. It may be better to bail out immediately before you do any lasting damage.

The `Environment` class provides a `FailFast` method. If you call this, the CLR will then terminate your application. (If you're running on Windows, it will also write a message to the Windows Event Log and provide details to WER.) If the `DOTNET_DbgEnableMiniDump` environment variable is set to 1, this will create a *minidump*, a file that can subsequently be loaded by a debugger to inspect the program's state at the moment of failure. You can optionally supply a string and an exception. These details will be written to the standard error output, and will also be preserved in the minidump if one is created. On Windows, these details will also be written to the event log, including the WER bucket values for the point at which the exception was thrown.

Exception Types

When your code detects a problem and throws an exception, you need to choose which type of exception to throw. You can define your own exception types, but the runtime libraries define a large number of exception types, so in a lot of situations, you can just pick an existing type. There are hundreds of exception types, so a full list would be inappropriate here; if you want to see the complete set, the online documentation for the

`Exception` class lists the derived types. However, there are certain ones that it's important to know about.

The runtime libraries define an `ArgumentException` class, which is the base of several exceptions that indicate when a method has been called with bad arguments. [Example 8-9](#) used `ArgumentOutOfRangeException`, and [Example 8-10](#) indirectly threw an `ArgumentNullException`, both of which derive from `ArgumentException`. It defines a `ParamName` property, which contains the name of the parameter that was supplied with a bad argument. This is important for multiargument methods, because the caller will need to know which one was wrong. All these exception types have constructors that let you specify the parameter name, and you can see one of these in use in [Example 8-9](#). The base `ArgumentException` is a concrete class, so if the argument is wrong in a way that is not covered by one of the derived types, you can just throw the base exception, providing a textual description of the problem.

Besides the general-purpose types just described, some APIs define more specialized derived argument exceptions. For example, the `System.Globalization` namespace defines an exception type called `CultureNotFoundException` that derives from `ArgumentException`. You can do something similar, and there are two reasons you might want to. If there is additional information you can supply about why the argument is invalid, you will need a custom exception type so you can attach that information to the exception. (`CultureNotFoundException` provides three properties describing aspects of the culture information for which it was searching.) Alternatively, it might be that a particular form of argument error could be handled specially by a caller. Often, an argument exception implies a programming error, but in situations where it might indicate an environment or configuration problem (e.g., not having the right language packs installed), developers might want to handle that specific issue differently. Using the base `ArgumentException` would be unhelpful in that case, because it would be hard to distinguish between the particular failure they want to handle and any other problem with the arguments.

Some methods may want to perform work that could produce multiple errors. Perhaps you're running some sort of batch job, and if some indi-

vidual tasks in the batch fail, you'd like to abort those but carry on with the rest, reporting all the failures at the end. For these scenarios, it's worth knowing about `AggregateException`. This extends the `InnerException` concept of the base `Exception`, adding an `InnerExceptions` property that returns a collection of exceptions.

TIP

If you nest work that can produce an `AggregateException` (e.g., if you run a batch within a batch), you can end up with some of your inner exceptions also being of type `AggregateException`. This exception offers a `Flatten` method, which recursively walks through any such nested exceptions and produces a single flat list with all the nesting removed. It returns an `AggregateException` with that list as its `InnerExceptions`.

Another commonly used type is `InvalidOperationException`. You would throw this if someone tries to do something with your object that it cannot support in its current state. For example, suppose you have written a class that represents a request that can be sent to a server. You might design this in such a way that each instance can be used only once, so if the request has already been sent, trying to modify the request further would be a mistake, and this would be an appropriate exception to throw.

Another important example is if your type implements `IDisposable` and someone tries to use an instance after it has been disposed. That's a sufficiently common case that there's a specialized type derived from `InvalidOperationException` called `ObjectDisposedException`.

You should be aware of the distinction between `NotImplementedException` and the similar-sounding but semantically different `NotSupportedException`. The latter should be thrown when an interface demands it. For example, the `IList<T>` interface defines methods for modifying collections but does not require collections to be modifiable—instead, it says that read-only collections should throw `NotSupportedException` from members that would modify the collection. An implementation of `IList<T>` can throw this and still be considered to be complete, whereas `NotImplementedException` means something is missing. You will most often see this in code generated by IDEs—these can create stub methods if you ask them to generate an interface

implementation or provide an event handler. They generate this code to save you from having to type in the full method declaration, but it's still your job to implement the body of the method. The generated methods will throw this exception so that you do not accidentally leave empty methods in place.

You would normally want to remove all code that throws

`NotImplementedException` before shipping, replacing it with appropriate implementations. However, there is a situation in which you might want to throw it. Suppose you've written a library containing an abstract base class, and your customers write classes that derive from this. When you release new versions of the library, you can add new methods to that base class. Now imagine that you want to add a new library feature for which it would seem to make sense to add a new abstract method to your base class. That would be a breaking change—existing code that successfully derives from the old version of the class would no longer work. You can avoid this problem by providing a virtual method instead of an abstract method, but what if there's no useful default implementation that you can provide? In that case, you might write a base implementation that throws a `NotImplementedException`. Code built against the old version of the library will not try to use the new feature, so it would never attempt to invoke the method. But if a customer tried to use the new library feature without overriding the relevant method in their class, they would then get this exception. In other words, this provides a way to enforce a requirement of the form: you must override this method if and only if you want to use the feature it represents. (You could use the same approach when adding new members to an interface with default implementations.)

There are, of course, other, more specialized exceptions in the framework, and you should always try to find an exception that matches the problem you wish to report. However, you will sometimes need to report an error for which the runtime libraries do not supply a suitable exception. In this case, you will need to write your own exception class.

Custom Exceptions

The minimum requirement for a custom exception type is that it should derive from `Exception` (either directly or indirectly). However, there are some design guidelines. The first thing to consider is the immediate base class: if you look at the built-in exception types, you'll notice that many of them derive only indirectly from `Exception`, through either `ApplicationException` or `SystemException`. You should avoid both of these. They were originally introduced with the intention of distinguishing between exceptions produced by applications and ones produced by .NET. However, this did not prove to be a useful distinction. Some exceptions could be thrown by both in different scenarios, and in any case, it was not normally useful to write a handler that caught all application exceptions but not all system ones, or vice versa. The class library design guidelines now tell you not to use these two base types.

Custom exception classes normally derive directly from `Exception`, unless they represent a specialized form of some existing exception. For example, we already saw that `ObjectDisposedException` is a special case of `InvalidOperationException`, and the runtime libraries define several more specialized derivatives of that same base class, such as `ProtocolViolationException` for networking code. If the problem you wish your code to report is clearly an example of some existing exception type, but it still seems useful to define a more specialized type, then you should derive from that existing type.

Although the `Exception` base class has a parameterless constructor, you should not normally use it. Exceptions should provide a useful textual description of the error, so your custom exception's constructors should all call one of the `Exception` constructors that take a string. You can either hardcode the message string³ in your derived class or define a constructor that accepts a message, passing it on to the base class; it's common for exception types to provide both, although that might be a waste of effort if your code uses only one of the constructors. It depends on whether your exception might be thrown by other code or just yours.

It's also common to provide a constructor that accepts another exception, which will become the `InnerException` property value. Again, if you're writing an exception entirely for your own code's use, there's not much point in adding this constructor until you need it, but if your exception is part of a reusable library, this is a common feature. [Example 8-13](#) shows a hypothetical example that offers various constructors, along with an enumeration type that is used by the property the exception adds.

Example 8-13. A custom exception

```
public class DeviceNotReadyException : InvalidOperationException
{
    public DeviceNotReadyException(DeviceStatus status)
        : this("Device status must be Ready", status)
    {
    }

    public DeviceNotReadyException(string message, DeviceStatus status)
        : base(message)
    {
        Status = status;
    }

    public DeviceNotReadyException(string message, DeviceStatus status,
                                    Exception innerException)
        : base(message, innerException)
    {
        Status = status;
    }

    public DeviceStatus Status { get; }
}

public enum DeviceStatus
{
    Disconnected,
    Initializing,
    Failed,
    Ready
}
```

The justification for a custom exception here is that this particular error has something more to tell us besides the fact that something was not in a suitable state. It provides information about the object's state at the moment at which the operation failed.

Unhandled Exceptions

Earlier, you saw the default behavior that a console application exhibits when your application throws an exception that it does not handle. It displays the exception's type, message, and stack trace and then terminates the process. This happens whether the exception went unhandled on the main thread or a thread you created explicitly, or even a thread pool thread that the CLR created for you.

The CLR provides a way to discover when unhandled exceptions reach the top of the stack. The `AppDomain` class provides an `UnhandledException` event, which the CLR raises when this happens on any thread.⁴ I'll be describing events in [Chapter 9](#), but jumping ahead a little, [Example 8-14](#) shows how to handle this event. It also throws an unhandled exception to try the handler out.

Example 8-14. Unhandled exception notifications

```
static void Main(string[] args)
{
    AppDomain.CurrentDomain.UnhandledException += OnUnhandledException;

    // Crash deliberately to illustrate the UnhandledException event
    throw new InvalidOperationException();
}

private static void OnUnhandledException(object sender,
    UnhandledExceptionEventArgs e)
{
    Console.WriteLine($"An exception went unhandled: {e.ExceptionObject}");
}
```

When the handler is notified, it's too late to stop the exception—the CLR will terminate the process shortly after calling your handler. The main

reason this event exists is to provide a place to put logging code so that you can record some information about the failure for diagnostic purposes. In principle, you could also attempt to store any unsaved data to facilitate recovery if the program restarts, but you should be careful: if your unhandled exception handler gets called, then by definition your program is in a suspect state, so whatever data you save may be invalid.

There is one important scenario in which unhandled exceptions will not terminate the process by default. If you use the `Task` class (described in [Chapter 16](#)) to run concurrent work, it catches any exceptions that emerge from that work, and puts the associated `Task` into a *faulted* state. This means that exceptions in tasks are, strictly speaking, never unhandled. However, if nothing ever inspects the state of a faulted task, that's called an *unobserved* exception. By default, this doesn't terminate the process, but .NET provides a mechanism (described in [Chapter 16](#)) by which you can discover when it happens.

Some application frameworks provide their own ways to deal with unhandled exceptions. For example, UI frameworks (e.g., Windows Forms, WPF) for desktop applications for Windows do this, partly because the default behavior of writing details to the console is not very useful for applications that don't show a console window. These applications need to run a message loop to respond to user input and system messages. It inspects each message and may decide to call one or more methods in your code, in which case it wraps each call in a `try` block so that it can catch any exceptions your code may throw. The frameworks may show error information in a window instead. And web frameworks, such as ASP.NET Core, need a different mechanism: at a minimum, they should generate a response that indicates a server-side error in the way recommended by the HTTP specification.

This means that the `UnhandledException` event that [Example 8-14](#) uses may not be raised when an unhandled exception escapes from your code, because it may be caught by a framework. If you are using an application framework, you should check to see if it provides its own mechanism for dealing with unhandled exceptions. For example, ASP.NET Core applications can supply a callback to a method called `UseExceptionHandler` during application startup. WPF has its own `Application` class, and its

`DispatcherUnhandledException` event is the one to use. Likewise, Windows Forms provides an `Application` class with a `ThreadException` member.

Even when you're using these frameworks, their unhandled exception mechanisms deal only with exceptions that occur on threads the frameworks control. If you create a new thread and throw an unhandled exception on that, it would show up in the `AppDomain` class's `UnhandledException` event, because frameworks don't control the whole CLR.

Summary

In .NET, errors are usually reported with exceptions, apart from in certain scenarios where failure is expected to be common and the cost of exceptions is likely to be high compared to the cost of the work at hand. Exceptions allow error-handling code to be separate from code that carries out work. They also make it hard to ignore errors—unexpected errors will propagate up the stack and eventually cause the program to terminate and produce an error report. `catch` blocks allow us to handle those exceptions that we can anticipate. (You can also use them to catch all exceptions indiscriminately, but that's usually a bad idea—if you don't know why a particular exception occurred, you cannot know for certain how to recover from it safely.) `finally` blocks provide a way to perform cleanup safely regardless of whether code executes successfully or encounters exceptions. The runtime libraries define numerous useful exception types, but if necessary, we can write our own.

In the chapters so far, we've looked at the basic elements of code, classes and other custom types, collections, and error handling. There's one last feature of the C# type system to look at: a special kind of object called a *delegate*.

¹ Strictly speaking, the CLR allows any type as an exception. However, C# can throw only `Exception`-derived types. Some languages let you throw other types, but it is strongly discouraged.

- 2** Exception filters cannot use the `await` keyword, which is discussed in [Chapter 17](#).
- 3** You could also consider looking up a localized string with the facilities in the `System.Resources` namespace instead of hardcoding it. The exceptions in the runtime libraries all do this. It's not mandatory, because not all programs run in multiple regions, and even for those that do, exception messages will not necessarily be shown to end users.
- 4** Although .NET does not support the creation of new appdomains, it still provides the `AppDomain` class, because that exposes certain important features, such as this event. It will provide a single instance via `AppDomain.CurrentDomain`.